

Present Value Engine Implementation Walkthrough

How the Go port values streams of money across time: field-presence classification, the geometric-series summation, the backward solvers, COLA, and variable-rate discounting.

Scope. The Go PV package `internal/finance/presentvalue/` (`calc.go`, `backward.go`, `variablerate.go`, `types.go`), the API wiring in `internal/api/handlers.go`, and the shared math.

Audience. Engineers maintaining or extending the Go port. Every claim is anchored to a `file:line` reference in `internal/`.

Lineage. A faithful port of `PRESVALU.pas` / `PVLXSCRN.pas`; for the original algorithm see `legacy/code_docs/present_value_dos_walkthrough.pdf`. The life-contingency integration is covered in the companion actuarial Go walk-through; here it is noted only where it hooks in.

Contents

- 0 How to read this document**
- 1 Entry & data model**
- 2 Classification & dispatch**
- 3 The summation mathematics**
 - 3.1 The geometric kernel · 3.2 PeriodicSummation · 3.3 COLA modes
- 4 Forward calc & the rate / as-of solves**
- 5 Backward solvers**
- 6 The variable-rate engine**
- 7 Math primitives & boundary cases**
- 8 Appendix: call graph & file:line index**

0 How to read this document

The Go PV engine is a worksheet that values streams of money. It has three editable blocks — **lump-sum** single payments, **periodic** streams (with optional COLA escalation), and a **rate** / **present-value** line carrying the discount rate and the as-of date. The caller leaves one cell blank and the engine solves for it. A “variable-rate” mode replaces the single discount rate with a schedule of rates over time.

Citation convention

References look like `calc.go:90` and resolve under `internal/finance/presentvalue/` unless another path is named (e.g. `api/handlers.go:1317`). Pascal lineage is noted as `(PRESVALU.pas:NNN)`.

The one formula to know. A periodic stream's PV is $\text{amt} \times \text{PeriodicSummation}$, and that factor is a geometric series $(1-f^n)/(1-f)$ whose ratio $f = e^{(\text{cola}-\text{rate})/P}$ nets escalation against discounting. When $\text{cola} = \text{rate}$, $f \rightarrow 1$ and the series collapses to n — the load-bearing edge case.

1 Entry & data model

The HTTP entry is `HandlePVCalc` [api/handlers.go:1317](#), decoding a `PVRequest` [api/handlers.go:344](#) whose optional scalars are **pointers** — omit `Rate` or `AsOfDate` to solve for it; supply `SumValue` to enable backward mode. Each non-nil pointer stamps a status; a nil pointer leaves `StatusEmpty` [api/handlers.go:1348–1449](#). Two subtleties: the rate is set to `StatusFromRate` (tested via `> StatusEmpty`, not `≥ InOutDefault`) [api/handlers.go:1361](#), and a non-empty `RateSchedule` switches the engine into variable-rate mode. The handler then calls `Calculate(input)` [api/handlers.go:1499](#).

The model [types.go](#): `LumpSumPayment` (date / amount / value + contingency), `PeriodicPayment` (from / to / per-yr / amount / COLA / value, plus the per-payment `Installments` used under a life contingency), `PresValLine` (as-of, a `RateEntry` holding the *continuous* rate, and the total `SumValue`), and `RateLine` (the variable-rate schedule: date + continuous rate). Status codes [interest/constants.go:53](#) use the `≥ InOutDefault` = “known” rule; a per-row line-status code [interest/constants.go:174](#) (`LineFullySpecified=254`, `LineContainsUnknown=253`, ...) is computed by counting present fields.

2 Classification & dispatch

Calculate `calc.go:297` runs three pre-dispatch checks — a two-life-contingency guard, the variable-rate branch (if `RateSchedule` is set, `FirstPass` is skipped entirely), and an unknown-POD shortcut — then `FirstPass` `backward.go:163` classifies each block, assigning a line-status and resolving the `Frontward/Backward` flags. `Frontward` is true only if rate + as-of are present and every populated row is fully specified; `Backward` fires when a row-level target exists and selects a `BackwardKind` by which field is blank:

Blank field	BackwardKind	Row / path	Set at
lump amount	PV-1	date + value given	<code>backward.go:433</code>
lump date	PV-2	amount + value given	<code>backward.go:439</code>
periodic amount	PV-4	both dates + value	<code>backward.go:462</code>
periodic to / from date	PV-5 / PV-6	one date + amount + value	<code>backward.go:468</code>
rate	PV-8	asof + rows + SumValue	<code>backward.go:492</code>
as-of date	PV-9	rate + rows + SumValue	<code>backward.go:500</code>

The dispatch decision `calc.go:347–356`: both flags → “more than one missing field” error; `Backward` → `BackwardCalc` (a switch on `BackwardKind`); else `forwardOnly`. (PV-3/PV-7 are intentionally absent — dead DOS V_3 code.)

3 The summation mathematics

Throughout: r = discount rate, c = COLA, P = `RealPerYr(peryr)`, n = installments. `YearsDif(asof, date)` is negative for future payments, so the exponential discounts.

3.1 The geometric kernel

`SumFormula(lnf, n)` [calc.go:20](#) is the closed form of $\sum_{i=0}^{n-1} e^{i \cdot \ln f}$:

$$\text{SumFormula} = (1 - e^{n \cdot \ln f}) / (1 - e^{\ln f}), \quad \rightarrow n \text{ when } |\ln f| < \text{teeny}$$

The teeny (1e-10) cutoff collapses the series to n equal terms; a tiny (1e-5) band swaps in 2nd-order Taylor expansions of $1 - e^x$ to avoid cancellation near zero [calc.go:21–49](#).

3.2 PeriodicSummation

`PeriodicSummation` [calc.go:90](#) sets the per-period log-ratio $\ln f = (c - r) / P$ [calc.go:94](#) (so $f = e^{\ln f}$), guards an infinite series ($\ln f \geq 0$ with a “forever” to-date $\rightarrow$ error), and returns the closed form:

$$\text{PeriodicSummation} = e^{r \cdot \text{since}} \cdot \text{SumFormula}(\ln f, n) \cdot [\text{phase factor}]$$

where `since` measures from the as-of date to the stream's reference point. The branch chooses whether to measure from a “standard loan date” one period before the first payment (`asof ≤ fromDate`, with an $e^{-r/P}$ phase correction) or from the to-date [calc.go:155–205](#).

The cola = rate degeneracy. When $c = r$, $\ln f = 0$, $f = 1$, and the geometric ratio is $0/0$. The teeny branch collapses it to n equal terms [calc.go:159](#): every escalated payment discounts to the same value, so the stream is worth n payment PVs. Every backward solver special-cases this.

3.3 COLA modes

COLA is entered as a *yield*. The continuous path converts it to a continuous rate $\text{cola} = \log(1 + \text{cola})$ and folds it into $\ln f$ [calc.go:126–129](#). The stepped path (`periodicSumAnnualCOLA` [calc.go:243](#)) instead multiplies the payment by $(1 + \text{cola})$ once per anniversary — kept consistent so the two agree at integer years. An exact (period-by-period) mode [calc.go:131–153](#) walks each payment date, used for weekly/biweekly, life-contingent, or explicitly-exact streams.

4 Forward calc & the rate / as-of solves

forwardOnly [calc.go:460](#) requires rate + as-of present. A lump value is $\text{amt} \cdot e^{\text{rate} \cdot \text{YearsDif}(\text{asof}, \text{date})}$
[calc.go:62](#); a periodic value is $\text{amt} \cdot \text{PeriodicSummation}$; sumValue aggregates all rows. Life-contingent rows multiply by LifeProb and the POD value is added — see the actuarial walk-through.

Unknown rate (PV-8) [backward.go:1187](#): a damped secant on the PV residual — $\text{diff} = (\text{target} - \text{sum}) \cdot \text{diff} / (\text{sum} - \text{oldSum})$ clamped to ± 0.04 , first guess 0.1, restart from 0, ≤ 30 iterations to $|\text{diff}| < \text{teeny}$. **Unknown as-of date (PV-9)** [backward.go:1262](#): a near-closed-form fixed point $\text{asof} += \ln(\text{target}/\text{sum})/\text{rate}$, ≤ 10 iterations to < 0.002 years. Both share evaluatePVAt [backward.go:1336](#), the non-mutating forward valuation kernel (which includes POD).

5 Backward solvers

Every row solver derives a **target** by subtracting the known rows (and POD) from SumValue via `computeKnownRowSum` `backward.go:581`.

Path	Solve	Method	Line
PV-1	lump amount	direct: $\text{amount} = \text{val} \cdot e^{\text{rate} \cdot \text{YearsDif}(\text{date}, \text{asof})}$ (algebraic inverse)	<code>backward.go:641</code>
PV-2	lump date	Newton on $\text{amt} \cdot e^{\text{rate} \cdot \text{yrs}}$; ≤ 30 iters. Rejected for life-contingent rows (survival depends on the unknown date)	<code>backward.go:706</code>
PV-4	periodic amount	direct: $\text{amount} = \text{val} / \text{weightedPeriodicFactor}$	<code>backward.go:817</code>
PV-5 / PV-6	periodic to / from date	analytic first estimate, then <code>refinePeriodicDate</code> snaps to a true installment (To-Date on the period grid; From-Date bisected on the calendar day)	<code>backward.go:888</code>

The date solvers reuse the exact forward factor (`weightedPeriodicFactor`, `evaluatePVAt`) they invert, so round-trips hold. A one-cent amount correction absorbs the residual of forcing a whole-period count `backward.go:1028`.

6 The variable-rate engine

A rate schedule replaces a single scalar r with a piecewise-constant function; the discount becomes $\int r(s) ds$ rather than $\text{rate} \cdot \text{YearsDif}$. `integrateRateForward` [variableRate.go:73](#) walks time, accruing $\text{rate} \cdot \text{YearsDif}$ within each band (the first entry's rate runs from $-\infty$). `VRDiscountFactor` [variableRate.go:124](#) is the per-payment multiplier:

$$\text{paymentDate} > \text{asof: } e^{-\int_{\text{asof}}^{\text{pay}} r} \text{ (discount)} \cdot \text{paymentDate} < \text{asof: } e^{+\int_{\text{pay}}^{\text{asof}} r} \text{ (accumulate)}$$

`vrPeriodicValue` [variableRate.go:159](#) walks each installment applying COLA and the discount factor (and a `LifeProb` weight under a contingency). `forwardVariableRate` [variableRate.go:273](#) is the entry; it requires every row fully specified (no per-row backward solve). The VR backward solves exploit linearity/monotonicity: `solveVariableRateAmount/POD` probe two forward runs and divide [variableRate.go:434, 495](#); `solveVariableRateDate` brackets a sign change and bisection on the calendar day against the true VR valuation [variableRate.go:582](#). Variable-rate mode *never* uses the closed-form `SumFormula` — it always walks period-by-period and integrates the schedule.

7 Math primitives & boundary cases

Exxp/Lnn [interest/math.go:43 / 67](#) (safe exp/ln; overflow at $x > 70$ is the “infinite present value” guard), RealPerYr/YieldFromRate/RateFromYield [interest/rates.go:42 / 63 / 79](#), and YearsDif/AddPeriod/AddYears [dateutil/dateutil.go:632 / 434 / 378](#). Boundary guards: teeny degeneracy/convergence cutoffs; an explicit infinite-series error when $\ln f \geq 0$ with a “forever” to-date [calc.go:97](#); the AddYears $|yrs| > 128$ cap that backstops a runaway date solve [dateutil/dateutil.go:379](#); and per-solver iteration caps (lump-date 30, rate 30×2 , as-of 10, date refinement 60+40).

8 Appendix: call graph & file:line index

HandlePVCalc	api/handlers.go:1317
└ build PVInput (pointer → status) → Calculate	calc.go:297
└─ checkSecondLifeProvided	calc.go:304
└─ [RateSchedule ≠ ∅] variable-rate branch	calc.go:311
└─┬ solveVariableRate POD/Amount/Date	variablerate.go:495 / 434 / 582
└─┬ forwardVariableRate	variablerate.go:273
└─┬┬ VRDiscountFactor → integrateRateForward	variablerate.go:124 / 73
└─┬ [PODUnknown] solveUnknownPOD	calc.go:412
└─ FirstPass (classify; Frontward/Backward + Kind)	backward.go:163
└─┬ Backward → BackwardCalc	backward.go:519
└─┬┬ computeKnownRowSum (target - known - POD)	backward.go:581
└─┬┬ PV-1/2 lump amount/date	backward.go:641 / 706
└─┬┬ PV-4 periodic amount	backward.go:817
└─┬┬ PV-5/6 periodic dates → refinePeriodicDate	backward.go:888 / 1058
└─┬┬ PV-8 rate (secant) → evaluatePVAt	backward.go:1187 / 1336
└─┬┬ PV-9 as-of (fixed point)	backward.go:1262
└─ forwardOnly	calc.go:460
└─┬ LumpSumValue $\text{amt} \cdot e^{\{\text{rate} \cdot \text{YearsDif}\}}$	calc.go:62
└─┬ PeriodicSummation → SumFormula annual-COLA exact loop	calc.go:90 / 20 / 243

Symbol	Reference
HandlePVCalc / Calculate	api/handlers.go:1317 · calc.go:297
FirstPass / BackwardCalc	backward.go:163 / 519
SumFormula / PeriodicSummation / annual COLA	calc.go:20 / 90 / 243
forwardOnly / evaluatePVAt	calc.go:460 · backward.go:1336
PV-1/2/4/5-6/8/9 solvers	backward.go:641 / 706 / 817 / 888 / 1187 / 1262
VRDiscountFactor / vrPeriodicValue / forwardVariableRate	variablerate.go:124 / 159 / 273

Per%Sense Go port. Citations resolve under internal/. For the original DOS algorithm, see legacy/code_docs/present_value_dos_walkthrough.pdf. The life-contingency integration is covered in the actuarial Go walk-through.